Project Proposal

A Web-Based Platform for Deploying ChatGPT-Native Applications

Course: Workshop on Intelligent Systems and Autonomous Agents
Instructor: Dr. Gorge Kor

Submitted by:

- Shalev Yosef
- Nadav Simon
- Dmitry Todoseyev

Institution: The Academic College Of Tel Aviv Yaffo
Submission Date: 21.12.25

1. Introduction

In recent years, large language models (LLMs) and conversational AI systems have become increasingly accessible to developers and businesses. Among these, ChatGPT has emerged as a widely adopted platform for building interactive, AI-powered experiences. However, deploying a customized ChatGPT-based application still requires technical knowledge, infrastructure management, and familiarity with APIs and deployment workflows.

This project proposes the design and development of a web-based platform that enables users to create and deploy customized ChatGPT-native applications without building the underlying infrastructure from scratch. The system provides a centralized interface for configuration, management, and deployment, while abstracting away much of the technical complexity.

At its current stage, the applications generated by the system are deployed and executed **within the ChatGPT environment itself**, rather than as standalone external applications. This design choice reflects existing platform and API limitations, and is explicitly addressed as part of the project scope.

2. Project Description and Goals

The goal of this project is to design a full system that allows users to configure, generate, and deploy ChatGPT-based applications through a web interface. The platform acts as an intermediary between the user and the OpenAI API, using an MCP (Model Context Protocol) server as a mediation layer.

The primary objectives of the project are:

- To provide a simple and accessible interface for deploying ChatGPT-native applications.
- To reduce the technical barrier for users who wish to create AI-powered chat applications.
- To support template-based customization of application appearance and behavior.
- To demonstrate a complete system architecture while acknowledging platform-level constraints.

3. Target Users and Use Cases

The platform is designed for a broad audience, including:

- Developers who want to quickly prototype ChatGPT-based applications.
- Businesses or individuals interested in deploying AI chat experiences.
- Users with limited technical background who want to experiment with AI applications.
- Early adopters exploring ChatGPT-native tools within the existing ecosystem.

No prior expertise in AI infrastructure or backend development is assumed.

4. System Overview and Conceptual Architecture

The system follows a modular, high-level architecture designed to separate concerns and simplify deployment.

At a conceptual level, the architecture includes the following components:

- **Web Platform (Frontend):**
  Provides user access, configuration options, and template selection.

- **Backend Logic:**
  Manages user sessions, subscriptions, and configuration data.

- **OpenAI API Integration:**
  The OpenAI API is used to create and configure the MCP server. The MCP server fetches relevant data from the web platform and injects it into predefined templates, which are then executed within the ChatGPT application environment.

- **MCP Server:**
  Acts as a mediation layer that translates user configurations into structured requests compatible with ChatGPT execution.

- **ChatGPT Environment:**
  The runtime environment where the generated applications are executed and demonstrated.

At this stage, all generated applications are **ChatGPT-native**, meaning they run inside the ChatGPT ecosystem rather than as standalone deployed services.

5. Design and User Experience

The platform emphasizes simplicity and usability. Rather than requiring users to design applications from scratch, the system relies on predefined templates that control layout and interaction style.

Key design principles include:

- Clear onboarding and guided configuration.
- Minimal technical terminology exposed to the user.
- Template-based customization rather than free-form design.
- A landing-page-style selection flow for application appearance.

The design is intentionally constrained to ensure feasibility within the project timeline and platform limitations.

6. Core System Components

The main functional components of the system include:

- **User Management:**
  Registration, authentication, and access control.
- **Subscription and Access Logic:**
  Basic handling of usage levels or feature availability.
- **Application Configuration Interface:**
  Allows users to select templates and define basic parameters.
- **Deployment Flow:**
  Generates and deploys a ChatGPT-integrated demo application.
- **Monitoring and Management:**
  Enables users to view and manage their created applications.

These components together form a complete system, even though certain features are intentionally limited.

7. Limitations and External Dependencies

A key aspect of this project is its reliance on external platforms and APIs. The system is constrained by:

- OpenAI API exposure and usage limitations.
- Restrictions on how applications can be executed within ChatGPT.
- Lack of full control over deployment outside the ChatGPT ecosystem.
- Scalability and performance considerations dependent on third-party services.

These limitations are not treated as flaws, but rather as realistic constraints that shape the system's scope and design decisions.

8. Development Plan and Estimated Timeline

The project development is divided into several phases:

1. **Requirements Analysis and System Design**
   Definition of scope, architecture, and responsibilities.
2. **Frontend Development**
   Implementation of the web interface and user flows.
3. **Backend and Integration Logic**
   Configuration handling and API mediation.
4. **Deployment and Testing**
   Validation of ChatGPT-native application generation.
5. **Documentation and Final Evaluation**
   System documentation and preparation for presentation.

As a group project, tasks are distributed among team members to ensure parallel progress.

9. Expected Outcomes and Future Extensions

By the end of the project, a functional web-based platform will be delivered, capable of generating and deploying ChatGPT-integrated demo applications.

Potential future extensions include:

- Advanced customization options.
- Integration with additional AI models or platforms.
- Enhanced analytics and usage monitoring.

These extensions are beyond the current scope but demonstrate the system's growth potential.

10. Conclusion

This project presents a practical and forward-looking approach to simplifying the deployment of ChatGPT-based applications. By focusing on ChatGPT-native execution and template-driven customization, the system balances feasibility with innovation. Despite external constraints, the project demonstrates a complete system design and provides a foundation for future expansion as platform capabilities evolve.